

HMI Copilot

“Build the whole plant: an overview plus a screen for each area, with pump status, flows, levels and alarms”

11

screens

560

objects

232

alarms

1,248

tags bound

0

by hand

And the file opens in EcoStruxure Operator Terminal Expert 4.4.

It was never saved by the product.

Today this is done by hand, and it is done twice

The work	What it looks like
Manual screen development	Hand-place every lamp, bargraph and trend. Then repeat it for 40 near-identical equipment screens.
Expert-driven configuration	Only senior engineers know which object, animation and colour standard applies. Juniors block on them.
Complex tag integration	Hundreds of PLC symbols mapped by hand onto object properties, one property sheet at a time.
Inconsistency	Two engineers on one project produce two differently-named, differently-coloured HMIs.

A wrong binding is not a drawing error. It is pump 2’s flow under pump 1’s label, found during an upset.

WHY THIS IS POSSIBLE AT ALL

The project format is fully legible

```
<project>.eote          ZIP    (nested entries use BACKSLASH separators)
|-- Project.dat, Target.dat  JSON   panel model, resolution
|-- Variables.db           SQLite the tags
|-- Alarm.db               SQLite alarm groups and alarms
|-- Bindings.dat           JSON    Sources[] -> Bindings[] -> Targets[]
+-- Screens\<guid>\Screen.dat JSON    the object tree
```

Plain JSON and plain SQLite. No signature. No encryption. Nothing to defeat.

So the first thing we did was not design a UI or pick a model. It was to take the product's own shipped templates apart and find out whether a project could be written from scratch at all.

It can. Everything after this follows from that.

LIVE

One sentence becomes a plant

DEMO · 1

Eleven screens, laid out as a board. A plant overview, then a screen per area, with navigation between them.

A 1,248-tag plant is not one screen — it is a screen hierarchy, and inferring that hierarchy from tag names is the real problem. Every unit gets a screen. A chiller is not an air compressor.

The screenshot displays the Schneider Electric HMI Copilot interface for a project named "Pump_Station". The interface is divided into several sections:

- Top Bar:** Includes the Schneider Electric logo, "Copilot" label, "Pump_Station" project name, a "saved" status, a resolution of "1024 x 600", a keyboard icon, and an "Export" button.
- Left Sidebar:** Contains a "Copilot" section with a "New" button and a list of screens: "Screen1 0", "PlantOverview 89", "PumpArea1 56", "PumpArea2 56", "InstrumentsArea1 53", "InstrumentsArea2 49", "Inst", and "+ Screen". Below this is a "Properties" panel with "Layers (560)", "Library", and "Tags (1248)".
- Main Canvas:** A grid of 11 screens laid out as a board. The screens are: "Screen1 0", "PlantOverview 89", "PumpArea1 56", "PumpArea2 56", "InstrumentsArea1 53", "InstrumentsArea2 49", "InstrumentsArea3 53", "InstrumentsArea4 49", "MotorArea1 56", "MotorArea2 56", and "MotorArea3 47".
- Right Panel:** A "Properties" panel with the text "Select an object on the canvas to inspect its properties and bindings."
- Bottom Bar:** Includes a "gemini" chat interface with a "revert to here" button, a "Ask for a change, or another screen..." input field, and a "8/8 Project named Pump_Station" status indicator.

At the bottom of the interface, there is a footer with the text "HMI Copilot v0.1 | Hackathon Build | 11 screens · 560 objects · 1248 tags" and a logo for "N".

LIVE

You watch it work, in engineering language

DEMO · 2

Eight steps, each reporting what it produced, each clickable to select the objects it created.

Not “Thinking...”. Parse PLC tags → 1,248. Infer equipment → pumps, instruments, motors. Configure alarms → 232. An engineer will not accept a screen they did not see being built.

The screenshot displays the Schneider Electric Copilot interface for HMI design. The top bar includes the Schneider Electric logo, 'Copilot' branding, and a 'saved' status. The main workspace is divided into several panels:

- Left Panel:** Contains a list of instructions or prompts, such as 'Build the whole plant: an overview plus a screen for each area, with pump status, flows, levels and alarms' and 'Building the plant overview and area screens based on the pump and flow tags.' Below this is a text input field labeled 'Describe the screen you need...' with a 'U' icon for undo.
- Top Bar:** Features a 'Design' tab, a 'Design' button, and a 'Simulate' button. It also shows a 'SCREENS' section with 'Screen1' selected and a '+ Screen' button.
- Right Panel:** Displays 'Properties', 'Layers (0)', 'Library', and 'Tags (1248)'. It includes a prompt: 'Select an object on the canvas to inspect its properties and bindings.'
- Canvas:** A large grid area for designing the HMI screen. A large, light gray rectangular area is currently visible on the canvas.
- Build Timeline:** A pop-up window titled 'Build timeline 0 of 8' showing a list of steps: 'Parse PLC tags' (running), 'Infer equipment', 'Select library objects', 'Generate screen layout', 'Configure alarms', 'Resolve tag bindings', 'Run validation', and 'Build export files'.

The bottom status bar indicates 'HMI Copilot v0.1 | Hackathon Build | 1 screen - 0 objects - 1248 tags' and the Schneider Electric slogan 'Accelerating a more sustainable and efficient world.'

LIVE

Every tag, wired and visible

DEMO · 3

The binding map, drawn from the real Sources / Targets graph the .eote contains.

This is the pain point the brief names, and today it is invisible until commissioning. Here it is a picture you can check before you export.

The screenshot displays the Schneider Electric HMI Copilot interface for a project named "Pump_Station_Demo". The interface is divided into several sections:

- Left Panel (TRY):** Contains a list of tasks for creating and configuring screens, such as "Create a screen for <equipment>, showing status and readings" and "Add a high alarm on <tag>".
- Bindings Map:** The central area shows a visual representation of the data bindings. It lists PLC tags (e.g., PMP_101_RUN, PMP_101_FLT, PMP_102_RUN, PMP_102_FLT, FT_101_PV, LT_101_PV, LT_101_HI) and maps them to object properties (e.g., Lamp_PUMP1_RUN.CurrentValue, AlarmGroup1.Alarm1.HiHi.VariableName). Green lines indicate display bindings, orange lines indicate alarm bindings, and red lines indicate unbound tags.
- Right Panel (Properties):** Shows the properties of the selected object, "Lamp #Lamp_PUMP1_RUN". It includes geometry (X, Y, W, H), data binding (PMP_101_RUN drives CurrentValue), and general settings (Type, Name, Unique id, Location, Width, Height, States).

At the bottom, a status bar indicates "HMI Copilot v0.1 | Hackathon Build | 1 screen · 19 objects · 7 tags" and a footer note reads "Accelerating a more sustainable and efficient world."

LIVE

Behaviour, before any hardware exists

DEMO · 4

Press Simulate and a process model drives the screen: lamps change state, numbers move, alarms raise.

Numerics are driven through their own alarm setpoints, so every alarm the project defines actually fires. Lead and standby pumps behave differently.

The screenshot displays the Schneider Electric HMI Copilot interface for a project named "Pump_Station_Demo". The interface is divided into several sections:

- Left Panel (Copilot):** Contains a "Describe what you need and keep going until it is right." section, a "TRY" section with suggestions like "Create a screen for <equipment>, showing status and readings", and a "WHAT A REQUEST NEEDS" section with instructions on naming screens, binding PLC tags, and setting panel size.
- Top Bar:** Shows the project name "Pump_Station_Demo", a "saved" status, and various tool icons. It also includes a "1 selected" indicator, a "Live · 8s" status, and a "4 active" indicator.
- Central Canvas:** Displays a grid with a "Pump Station 1" screen design. The design includes a "PUMPS" section with "PUMP 1 RUNNING" (green) and "PUMP 2 STOPPED" (grey) buttons, and a "PROCESS" section with "Flow" (57.1 LPM) and "Level" (102.4 %) readouts. Below these is an "ACTIVE ALARMS" table.
- Right Panel (Properties):** Shows the "Lamp" properties for "#Lamp_PUMP1_RUN", including geometry (X: 40, Y: 120, W: 200, H: 76), data binding (PMP_101_RUN drives CurrentValue), and general settings (Type: Lamp, Name: Lamp_PUMP1_RUN, Unique id: dabb5b1-b809-4576-b4bb-15d78108).

The "ACTIVE ALARMS" table shows the following data:

TIME	VARIABLE	MESSAGE	SEV	STATE
05:34:39	PMP_102_FLT	Pump 2 fault	5	ACTIVE
05:34:39	LT_101_PV	Tank level critically high	5	ACTIVE
05:34:39	LT_101_HI	Tank high level	3	ACTIVE
05:34:39	LT_101_PV	Tank level high	3	ACTIVE

The bottom of the interface shows the "HMI Copilot v0.1" version, "Hackathon Build", and "1 screen · 19 objects · 7 tags". A footer note reads "Accelerating a more sustainable and efficient world."

One model, two renderers — the preview cannot lie



The same JSON drives the preview and the file. There is no second model of the screen to drift from the first.

And it is the compiler that enforces it, not discipline: the canvas switches exhaustively over the part union, so it cannot draw a part the packager cannot write.

What the engineer approves is what OTE opens.

PROOF

Built, not proposed

Two .eote files written from scratch, both open in EcoStruxure Operator Terminal Expert 4.4. Neither was ever saved by the product.

19

parts placed

7

typed tags

5

alarms

11

bindings

0

placed by hand

467

tests

474

shipped symbols indexed

50

real part types extracted

2

independent packagers
that agree

A Python reference and a TypeScript packager, structurally diffed against each other. Two implementations agreeing is a stronger claim than either passing its own tests.

Who pays, and what it costs them

	Who	Why they buy	Price
The wedge	System integrators and panel builders	Fixed-price work. Time saved is margin on hours they already quoted, and they decide fast.	₹3,499 / seat / month
Expand	Plants, utilities, OEMs	They own the house style. Standards packs make every contractor produce their HMI.	from ₹25,00,000 / year
Scale	Schneider Electric	A generative front end for the format they already ship, with no change to the product.	OEM

#	BREAK-EVEN	Indian automation engineer, fully loaded	INR 700-1,200 / hour
		Team seat	INR 41,988 / year
		Hours it must give back	41,988 / 950 = ~44 h = 5.5 days

Does generating your first draft and resolving your bindings save you five days a year? That is the whole question — and it needs no savings claim we have not measured.

The AI does not replace the HMI engineer.

It encodes the senior engineer's judgement — which part to use, which alarm every pump needs, which naming standard applies — and gives it to everyone on the team, every time.

It is working today, at the file boundary, with no change to the product.

Ask us to open the generated project on your machine.